EN.601.447/647 Computational Genomics: Sequences

Homework 3 (Fall 2026)

Out of 25 points (15 written, 10 programming).

Programming questions

1. **Binary search on the suffix array**. (3 pts)

   Submit this question as `hw3q1`*, with the appropriate suffix:* `.py`*,* `.java` *or* `.cpp`*.*

   Write a program that uses binary search on the suffix array of T to decide if a pattern P occurs in T, and that reports every character comparison made along the way.

   The input file has three lines: a text T ending in `$`; the suffix array of T, as integers separated by single spaces; and a pattern string P. Apart from T's final `$`, both strings use only `A`, `C`, `G` and `T`. Assume T's length (counting `$`) is one less than a power of 2, e.g. 7, 15, 31, ... This way, every binary-search iteration will have a pivot in the exact middle. The program will be run like this:

   ```
   python3 hw3q1.py input.txt output.txt
   ```

   The algorithm must follow the binary search algorithm we described in lecture. At each pivot, compare P with the suffix one character at a time starting from $P[0]$, and stop comparing at the first mismatch. As soon as the algorithm finds that P matches, it should report that P occurs ("present") and stop. If the algorithm exhausts all binary search options without finding a match, it should stop and report that P does not occur ("absent").

   As your algorithm works, it should print one output line per character comparison in the order the comparisons are made. Each such line will consist of five outputs separated by spaces: (1) the number of the comparison, counting up from 1; (2) the offset in P; (3) the offset in T; (4) the character of P; and (5) the character of T. The last line of output is `present` or `absent` according to whether P matched.

   Example. Let $T =$ `TAGACA$`:

row	SA	suffix
0	6	`$`
1	5	`A$`
2	3	`ACA$`
3	1	`AGACA$`
4	4	`CA$`
5	2	`GACA$`
6	0	`TAGACA$`

   If `input.txt` contains

```
TAGACA$
6 5 3 1 4 2 0
ACA
```

then `output.txt` should contain

```
1 0 1 A A
2 1 2 C G
3 0 5 A A
4 1 6 C $
5 0 3 A A
6 1 4 C C
7 2 5 A A
present
```

This example is given as a visible Gradescope test.

2. **The BWT from the suffix array**. (2 pts)

   Submit this question as `hw3q2`*, with the appropriate suffix:* `.py`*,* `.java` *or* `.cpp`*.*

   Write a program that reads a text T and writes three lines: (1) the suffix array of T as integer offsets separated by single spaces; (2) BWT(T); and (3) the number of *runs* in BWT(T). A run is a maximal stretch of one repeated character. E.g. the string `AA$TTTT` has 3 runs: `AA`, `$` and `TTTT`.

   Compute the BWT from the suffix array, as in lecture: the BWT character in row i is the character just before (to the left of) the suffix at offset SA[i] in T. You may build the suffix array simply by sorting the suffixes, even though that's not very efficient. Offsets are 0-based.

   The input file holds T on one line: `A`, `C`, `G` and `T` followed by a single `$`, at most 10,000 characters in all. You would run the program like this:

```
python3 hw3q2.py input.txt output.txt
```

   Example. If `input.txt` contains

```
TAGACA$
```

   then `output.txt` should contain

```
6 5 3 1 4 2 0
ACGTAA$
6
```

   This is the text from Q1's example, and the suffix array is the one shown there. The six runs are `A`, `C`, `G`, `T`, `AA` and `$`. This example is given as a visible Gradescope test.

3. **Backward search with the BWT**. (5 pts)

Submit this question as `hw3q3`*, with the appropriate suffix:* `.py`*,* `.java` *or* `.cpp`*.*

Write a program that reads BWT(T) and a list of patterns, runs the "backward search" algorithm for each pattern following the algorithm we discussed in the FM index lecture, and reports how the range of Burrows-Wheeler matrix (BWM) rows narrows at each step for each pattern.

Recall that backward search processes P from right to left. At step 0 the range is all-inclusive, i.e. spans all rows of the BWM. After step k it spans just the rows beginning with the length-k suffix of P. Each new range is computed from the previous one using F, L and the ranks we discussed in lecture.

You do not need "checkpoints" here; it is fine to compute ranks on-the-fly, or to precompute them, even though those options are inefficient in big-genome scenarios.

At each step, report the characters of L spanned by the backward-search range. If the range becomes empty, report `-` for that step and stop.

Your program will take three command-line arguments: the BWT file containing BWT(T) on one line, the file with P, and the file where output will be written:

```
python3 hw3q3.py bwt.txt patterns.txt output.txt
```

The pattern file has one pattern per line. Each pattern has between 1 and 50 characters. The BWT will contain at most 2,000 characters and there will be at most 200 patterns. The output will have one line per pattern, in the order the patterns are given: the pattern, then what you report at steps 0, 1, 2 and so on, separated by single spaces.

Example. Let $T =$ `TAGACA$`, the text from Q1 and Q2. If `bwt.txt` contains BWT(T),

```
ACGTAA$
```

and `patterns.txt` contains

```
A
GA
ACA
CAT
```

then `output.txt` should contain

```
A ACGTAA$ CGT
GA ACGTAA$ CGT A
ACA ACGTAA$ CGT A G
CAT ACGTAA$ $ -
```

This example is given as a visible Gradescope test.

4. **Approximate suffix-prefix overlap**. (0 pts)

Note: this question (below) is **not due with Homework 3**. It will appear on Homework 4 instead.

When we assemble a genome from reads, we look for pairs of reads where the end of one overlaps the start of the other. Because of sequencing errors, the overlap may not be exact.

Write a program that reads two strings X and Y and finds the best overlap between a suffix of X and a prefix of Y. Score an alignment of a suffix of X with a prefix of Y by adding $+1$ for each position where the letters match and -2 for each mismatch, insertion or deletion. Give each prefix of Y the score of its best alignment with any suffix of X. The best overlap is the prefix with the highest score; if several prefix lengths tie, take the longest. The penalties mean that extending an overlap pays off only if the extra letters mostly match. The suffix and the prefix may be empty, and aligning the empty suffix with the empty prefix scores 0, so the best score is never negative. Write two numbers on one line: the length of the best prefix of Y, and its score.

The input file has X on its first line and Y on its second. Each has between 1 and 500 characters, all `A`, `C`, `G` or `T`. The command line is the same as in Q1.

Adapt the edit distance matrix from lecture, with X labeling the rows and Y the columns, so that each cell holds a best score instead of a fewest number of edits. Think about how to initialize the first row and the first column, given that the suffix of X may begin anywhere in X but the prefix of Y must begin at $Y[0]$, and about which cells describe alignments that reach the end of X.

Example. If `input.txt` contains

```
GGATCCATGCAT
CATGGATCTA
```

then `output.txt` should contain

```
7 4
```

The last three letters of X, `CAT`, match the first three of Y exactly, for a score of 3. But the prefix `CATGGAT` of Y aligns with the suffix `CATGCAT` of X with 6 matches and 1 mismatch, for a score of $6 - 2 = 4$, which is better. This example is given as a visible Gradescope test.

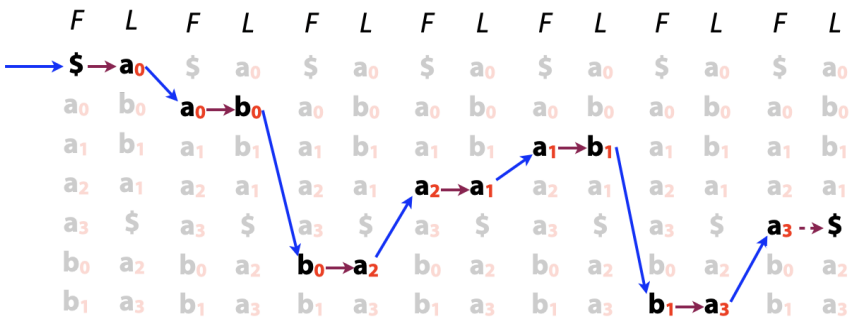
Note: this question (above) is **not due with Homework 3**. It will appear on Homework 4 instead.

Written questions

1. **BWT by hand**. (5 pts)

   (a) (1 pt) Give BWT(T) for T = `CATCAT$`.

   (b) (3 pts) Here is a different BWT, unrelated to part (a): `AGT$GAAAG`. Reverse it to find the T it came from. Show your work by drawing a figure like this one from the lecture slides, which reverses the BWT of `abaaba$`: the F and L columns with ranks, written once per step, with arrows tracing the reversal from the `$` row.



   Then write T. You don't have to put subscript ranks on the letters in the F and L columns; it's fine if you just write the letters themselves.

   (c) (1 pt) Now try the same reversal procedure on `GTA$T`. Say what goes wrong, and explain why this shows that `GTA$T` is not the BWT of any string. Two or three sentences are enough.

2. **BWT from column 3**. (3 pts)

   A string T has length 5, including the final `$`. The third column of its Burrows-Wheeler matrix (the middle one), read from top to bottom, is `BAB$A`. Assume `$` < `A` < `B`. What is T, and what is BWT(T)? Explain how you get your answer.

   Food for thought (ungraded): Is it always possible to reconstruct the original string T from a single column of its Burrows-Wheeler matrix ***other than*** *the last column?*

3. **Edit distance and traceback**. (4 pts)

   Let X = `GATTTG` label the rows and Y = `GTGTA` label the columns of an edit distance matrix, filled in as in lecture.

   (a) (2 pts) Fill in the whole matrix, and give the edit distance between X and Y.

   (b) (2 pts) There is exactly one optimal alignment of X and Y. Trace back from the bottom-right cell to find it. Write it out as two rows, with `-` for a gap, and say how many substitutions and how many gaps it has.

4. **Rotations, Hamming distance and edit distance**. (3 pts)

For a string X of length $n \geq 2$, let $R(X)$ be X rotated left by one: $R(X) = X[1]X[2]\ldots X[n-1]X[0]$. For example, $R(\texttt{GATTACA}) = \texttt{ATTACAG}$.

(a) (1 pt) Explain why the edit distance between X and $R(X)$ is at most 2, whatever X is.

(b) (1 pt) Give a DNA string X' of length 8 such that the Hamming distance between X' and $R(X')$ is 8.

(c) (1 pt) For your X' from (b), what is the edit distance between X' and $R(X')$ exactly? Explain why it cannot be smaller.

5. **Special review problem: backward search**. (0 pts)

Ungraded. A similar problem will be on Midterm 2. Solutions will be posted.

Below are F and L for T = `TACAGATACACAG$`. L has checkpoints at rows 0, 4, 8 and 12. A checkpoint at row i stores, for each character, how many times it occurs in L from row 0 through (and including) row i. If we are in a particular row and would like to know the number of letters occurring prior, we "walk" to the nearest checkpoint, which might be either upward or downward from our row. If two checkpoints are equally near, we walk downward. The suffix array sample keeps the entries for suffixes at T offsets that are multiples of 4.

row	F	L	checkpoint					SA sample
			`$`	`A`	`C`	`G`	`T`	
0	`$`	`G`	☐	☐	☐	☐	☐	
1	`A`	`T`						
2	`A`	`C`						
3	`A`	`T`						
4	`A`	`C`	☐	☐	☐	☐	☐	
5	`A`	`C`						
6	`A`	`G`						
7	`C`	`A`						8
8	`C`	`A`	☐	☐	☐	☐	☐	
9	`C`	`A`						
10	`G`	`A`						12
11	`G`	`A`						4
12	`T`	`A`	☐	☐	☐	☐	☐	
13	`T`	`$`						0

(a) Fill in the boxes: the counts of `$`, `A`, `C`, `G` and `T` stored at each checkpoint. Remember that the counts are cumulative and should include the character in the checkpoint's own row.

(b) Use backward search to find the rows beginning with P = `ACA`. Write the range of rows you obtain after each step as $[i, j)$, where i is the first row in the range and j is one more than the last row.

(c) The last step of (b) counts `A`s present in L up to this range (not including its first letter), and through the final letter of the range. For each of these two counts, report which checkpoint is used, and how the value in the checkpoint is used to get the count.

(d) For each occurrence of `ACA`, report how many LF steps are needed to get from the row that has that occurrence as a prefix, to a row for which we have a sampled suffix array entry. Also report the occurrence's offset in T. Again, please do this for all occurrences.